Module 5 Lab Session 1: Make the Database Talk

Field	Value
Module	M5: Entity Framework Core 10 and PostgreSQL
Session	1 of 3
Exercises	1 (DbContext configuration and migrations), 2 (LINQ queries and engine experiments)

Welcome to the Persistence Layer Sprint

In the previous module, you built the TMS Web API on top of an in-memory storage dictionary inside `EnrollmentService`. Yared, the lead instructor, opens this session with a simple demonstration: he enrolls a new student in a course, restarts the API process, and requests the student list. The enrollment is gone.

Volatile system memory is perfectly fine for understanding routing, dependency injection, and middleware. However, an enterprise system requires data that survives restarts, system failures, and upgrades. The database is the ultimate contract that the institution trusts, and Entity Framework (EF) Core 10 is the bridge that allows your C# codebase to communicate with this persistent store.

In this lab session, you will connect your existing `TmsApi` codebase to a local PostgreSQL instance. You will define your database tables using C# classes, write and inspect your first database migration, and run experiments to demystify how LINQ composition works under the hood. By the end of this session, your console will output the exact SQL commands generated by your queries, proving that filtering, sorting, and grouping happen where they belong: in the database, not in memory.

Before You Begin: Set Up PostgreSQL and Tooling

Before writing any C# code, ensure your local environment has PostgreSQL installed and the .NET entity framework command-line tools ready.

1. Verify PostgreSQL Installation

If you do not have PostgreSQL installed, download the installer from postgresql.org/download and complete the installation. Be sure to note the password you assign to the default `postgres` database user.

Ensure the PostgreSQL command-line utility is accessible by opening a terminal and running:

```
psql --version
```

If your system returns an error, make sure the PostgreSQL binary path (e.g., `C:\Program Files\PostgreSQL\18\bin` on Windows) is added to your environment `PATH` variable.

2. Confirm your .NET SDK

Verify that you are using .NET 10:

```
dotnet --version
```

This must return a `10.x.y` version. If it returns an older version, install the latest .NET 10 SDK before proceeding.

3. Add EF Core Packages to TmsApi

Open your terminal and navigate to the folder containing your existing `TmsApi` project from Module 4. Run the following commands to add the required Entity Framework Core packages:

```
dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add package Microsoft.EntityFrameworkCore.Design
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Before continuing, understand what these packages do:

Package	What it is	Why we need it
`Npgsql.EntityFrameworkCore.PostgreSQL`	The PostgreSQL Database Provider for EF Core.	EF Core is database-agnostic. This package translates your LINQ queries into PostgreSQL-compliant SQL commands and handles database connections.
`Microsoft.EntityFrameworkCore.Design`	Design-time logic for EF Core.	Provides compiler services that allow EF Core to inspect your project code structure to generate migrations. It does not ship with your compiled production binary.
`Microsoft.EntityFrameworkCore.Tools`	Package-manager console tooling hooks.	Integrates migration tools into the .NET development environment and is required for design-time CLI capabilities.

4. Verify the Entity Framework CLI Tools

Ensure the `dotnet-ef` CLI tool is installed globally on your machine.

What is the `dotnet-ef` tool? It is a command-line tool that acts as your migration operator. While the EF packages are referenced inside your C# code, the `dotnet-ef` tool is a command-line utility that compiles your project, detects model changes, and creates or runs migration scripts.

Verify if the tool is installed by running:

```
dotnet ef --version
```

If the tool is missing, install it globally by running:

```
dotnet tool install --global dotnet-ef
```

When installed, running `dotnet ef --version` should print a `10.x.y` version matching your SDK.

Exercise 1: Configure TmsDbContext and Apply the First Migration

Before you can query data using C#, you must model your database tables as C# classes (Entities) and group them into a context (`DbContext`) that manages connections, tracks state, and translates code to database calls.

> **A note on the models you sketched in M1.** Back in C# Essentials you modelled the full TMS domain — `Student`, `Course`, `Enrollment`, `Assessment`, and `Certificate` — inside the `TmsCore` console project, as immutable shapes (`record` types, `init`-only properties, string identifiers like `Code`). That was the right design for a console app teaching value semantics. Persistence changes the rules, so we evolve those shapes here rather than copy them verbatim: - **Mutable properties.** EF Core tracks each loaded row and writes back changes. It needs settable properties, so the entities below drop `init`/`record` immutability. - **Stable surrogate keys.** Relational tables want a stable, generated primary key rather than a business value like a course code that might change. We use `int Id` (PostgreSQL auto-increments it) because it is compact, fast to index, and easy to read in logs and `psql` — the right default for this course. GUIDs (`Guid` → PostgreSQL `uuid`) are a perfectly valid alternative when you need keys generated on the client, merged across databases, or non-guessable in URLs; the cost is 16 bytes per key and, for *random* GUIDs, poorer index locality — reach for time-ordered `Guid.CreateVersion7()` (UUIDv7) if you go that way. Either way, the key is a surrogate, so foreign keys (`StudentId`/`CourseId`) get a clean target instead of string codes. - **Navigation properties.** Relationships (`Enrollment` → `Student`/`Course`, and the `ICollection<Enrollment>` collections) are what let EF translate your LINQ joins and `GroupBy` into SQL in Exercise 2.
>
> So treat the classes below as the **persistence-ready evolution** of your M1 domain — same TMS concepts, reshaped for the database. Recreate them in `TmsApi` as shown (do not paste the old `TmsCore` versions, which will not migrate or track correctly).

Step 1: Define Your Database Entities

Create a folder named `Entities` in your `TmsApi` project. Add all five C# classes for the TMS domain below. You will wire the first three (`Student`, `Course`, `Enrollment`) into the database in this exercise; `Assessment` and `Certificate` are defined here too, but you will connect them yourself in the Extended Exercise so you practise the migration loop on your own.

1. **Entities/Student.cs**

```
namespace TmsApi.Entities;

public class Student
{
    public int Id { get; set; }                                  // sur
rogate primary key — internal, used by foreign keys
    public required string RegistrationNumber { get; set; } // na
tural key — human-readable (uniqueness configured in Session 2)
    public required string Name { get; set; }
    public decimal GPA { get; set; }
    public bool IsActive { get; set; } = true;

    // Navigation property for many-to-many relationship
    public ICollection<Enrollment> Enrollments { get; set; } = ne
w List<Enrollment>();
}
```

2. **Entities/Course.cs**

```
namespace TmsApi.Entities;

public class Course
{
    public int Id { get; set; }                // surrogate primar
y key — internal, used by foreign keys
    public required string Code { get; set; } // natural key — hu
man-readable (uniqueness configured in Session 2)
    public required string Title { get; set; }
    public int Capacity { get; set; }

    // Navigation property for many-to-many relationship
    public ICollection<Enrollment> Enrollments { get; set; } = ne
w List<Enrollment>();
}
```

3. **Entities/Enrollment.cs**

```
using System;

namespace TmsApi.Entities;

public class Enrollment
{
    public int Id { get; set; }
    public int StudentId { get; set; }
    public int CourseId { get; set; }
    public decimal? Grade { get; set; } // Nullable, as student m
ay be currently enrolled
    public DateTime EnrolledAt { get; set; } = DateTime.UtcNow;

    // Navigation properties back to entities
    public Student Student { get; set; } = null!;
    public Course Course { get; set; } = null!;
}
```

4. **Entities/Assessment.cs** — a quiz or practical task that belongs to one `Course`.

```
namespace TmsApi.Entities;

public class Assessment
{
    public int Id { get; set; }
    public required string Title { get; set; }
    public decimal MaxScore { get; set; }
    public decimal Weight { get; set; } // share of the final gra
de, e.g. 0.30m for 30%

    // Foreign key + navigation to the owning course
    public int CourseId { get; set; }
    public Course Course { get; set; } = null!;
}
```

5. **Entities/Certificate.cs** — issued to one `Student` for completing one `Course`.

```
using System;

namespace TmsApi.Entities;

public class Certificate
{
```

```
    public int Id { get; set; }                    // surrogate
 primary key
    public required string SerialNumber { get; set; } // natural
key — human-readable (uniqueness configured in Session 2)
    public DateTime IssuedAt { get; set; } = DateTime.UtcNow;

    // Foreign keys + navigation to the student and course
    public int StudentId { get; set; }
    public int CourseId { get; set; }
    public Student Student { get; set; } = null!;
    public Course Course { get; set; } = null!;
}
```

Two keys per entity, on purpose. Each of these tables has an `int Id` *surrogate* key and a *natural* key (`Course.Code`, `Student.RegistrationNumber`, `Certificate.SerialNumber`). Note the natural-key property does **not** repeat its class name it is `Code`, not `CourseCode` following the .NET guideline against stuttering members. (The `<Entity>Id` names you see, like `Enrollment.CourseId`, are different: those are foreign keys pointing at *another* entity, which is exactly EF Core's key-naming convention.) They do different jobs, and good schemas keep both: - The **surrogate `Id`** is what relationships use. It never changes and means nothing to a human, so foreign keys (`Enrollment.CourseId`, `Certificate.StudentId`, …) point at it and stay stable even if a course is renamed or recoded. - The **natural key** is what people read and type "CS-101", a student's registration number, a printed certificate serial. It *should* be unique so the database refuses duplicates, but we do *not* use it as the primary key, because business identifiers have a habit of changing or being reused, and you do not want every foreign key to break when they do.

For now these classes stay deliberately plain no database attributes. You will declare the **unique constraint** in Session 2 with the Fluent API (`builder.HasIndex(c => c.Code).IsUnique()`), which keeps the entity free of persistence concerns. So in this session's first migration the natural-key columns are created as `NOT NULL` (because they are `required`) but not yet unique; that index arrives with your Session 2 configuration.

Step 2: Implement TmsDbContext

Create a folder named `Data` in your `TmsApi` project. Add a class file named `Data/TmsDbContext.cs`:

```
using Microsoft.EntityFrameworkCore;
using TmsApi.Entities;

namespace TmsApi.Data;

public class TmsDbContext(DbContextOptions<TmsDbContext> options) : DbC
ontext(options)
{
    public DbSet<Student> Students => Set<Student>();
    public DbSet<Course> Courses => Set<Course>();
    public DbSet<Enrollment> Enrollments => Set<Enrollment>();
}
```

Step 3: Register the DbContext in Program.cs

Open your `Program.cs` file. Locate the section where services are registered, and add the DbContext registration using Npgsql:

```
using Microsoft.EntityFrameworkCore;
using TmsApi.Data;

// Register TmsDbContext scoped for incoming HTTP requests
builder.Services.AddDbContext<TmsDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("TmsDat
abase")));
```

Step 4: Configure Connection String

Open `appsettings.Development.json` and add the `ConnectionStrings` block at the root level, replacing `your_postgres_password` with your actual PostgreSQL password:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "ConnectionStrings": {
    "TmsDatabase": "Host=localhost;Database=TmsDb;Username=postgres;Pas
sword=your_postgres_password"
  }
}
```

Step 5: Generate the First Migration

From your project directory where your `csproj` file sits, run:

```
dotnet ef migrations add InitialCreate
```

This command analyzes your entity properties and builds a C# migration script detailing the SQL layout changes.

Step 6: Inspect the Generated Migration File

Before pushing changes to the database, open the newly created migration file located inside the `Migrations/` directory. Find the class and review the two main methods: - **`Up(MigrationBuilder migrationBuilder)`**: This method executes when applying the migration. Locate the `CreateTable` calls. Identify the primary keys and the foreign keys configured for `Enrollments`. Note that the natural-key columns (`Courses.Code`, `Students.RegistrationNumber`) are created `NOT NULL` but **not** unique yet you add that unique index in Session 2 with the Fluent API. - **`Down(MigrationBuilder migrationBuilder)`**: This method executes if you roll back the migration. Verify that it drops the tables in reverse dependency order (`Enrollments` first, then `Students` and `Courses`).

> **Why don't `Assessment` and `Certificate` show up when you migrate in a moment?** A C# class only becomes a database table once it is part of the EF Core *model* that is, registered as a `DbSet` (or reachable through a navigation property from an entity that is). In Step 2 you register only `Student`, `Course`, and `Enrollment`, and notice that `Student`/`Course` do **not** carry collections pointing at `Assessment` or `Certificate`. So EF Core never discovers those two classes, and your first migration creates three tables, not five. Wiring them in is your Extended Exercise.

Step 7: Apply the Migration

Apply the migration changes to your PostgreSQL instance:

```
dotnet ef database update
```

Step 8: Verify the Schema in PostgreSQL

Open your command terminal and log into your PostgreSQL instance to verify that the database and its tables were successfully provisioned:

```
psql -U postgres -d TmsDb -c "\dt"
```

You should see output indicating that the tables exist:

```
          List of relations
 Schema |    Name     | Type  |  Owner
--------+-------------+-------+----------
 public | Courses     | table | postgres
```

```
 public | Enrollments | table | postgres
 public | Students    | table | postgres
(3 rows)
```

Exercise 2: The LINQ Engine Logging, Deferred Execution, and Translation Limits

LINQ queries look like standard C# code, but when run against a `DbSet`, they act as instructions to compile SQL. In this exercise, you will configure logging, seed data, and run experiments to see exactly when and how C# translates to database commands.

Step 1: Enable Console SQL Logging

To see the exact SQL running behind the scenes, update your `AddDbContext` registration in `Program.cs` to print logs directly to the command window during development:

```
builder.Services.AddDbContext<TmsDbContext>(options =>
    options.UseNpgsql(builder.Configuration.GetConnectionString("TmsDat
abase"))
           .LogTo(Console.WriteLine, LogLevel.Information) // Log SQL t
o output window
           .EnableSensitiveDataLogging()); // Show parameters in query
logs (dev only)
```

Step 2: Write an Auto-Seeder

To ensure your database contains records to query, add this temporary seeding block inside `Program.cs` just before `app.Run()`. It verifies if database tables are empty at application startup and populates them:

```
// Seed test data at startup
using (var scope = app.Services.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<TmsDbContex
t>();
    context.Database.Migrate(); // Applies any pending migrations; keep
s migration history intact

    if (!context.Students.Any())
    {
        var students = new List<Student>
        {
            new() { RegistrationNumber = "TMS-2026-0001", Name = "Alice
```

```
 Smith", GPA = 3.8m, IsActive = true },
            new() { RegistrationNumber = "TMS-2026-0002", Name = "Bob J
ones", GPA = 2.9m, IsActive = true },
            new() { RegistrationNumber = "TMS-2026-0003", Name = "Charl
ie Brown", GPA = 3.4m, IsActive = false },
            new() { RegistrationNumber = "TMS-2026-0004", Name = "Diana
 Prince", GPA = 3.9m, IsActive = true },
            new() { RegistrationNumber = "TMS-2026-0005", Name = "Evan
Wright", GPA = 2.5m, IsActive = true }
        };
        context.Students.AddRange(students);

        var courses = new List<Course>
        {
            new() { Code = "CS-101", Title = "Introduction to Computer
Science", Capacity = 30 },
            new() { Code = "CS-201", Title = "Data Structures and Algor
ithms", Capacity = 25 },
            new() { Code = "MAT-101", Title = "Calculus I", Capacity =
40 }
        };
        context.Courses.AddRange(courses);
        context.SaveChanges();

        var enrollments = new List<Enrollment>
        {
            new() { StudentId = students[0].Id, CourseId = courses[0].I
d, Grade = 4.0m },
            new() { StudentId = students[0].Id, CourseId = courses[1].I
d, Grade = 3.6m },
            new() { StudentId = students[1].Id, CourseId = courses[0].I
d, Grade = 2.8m },
            new() { StudentId = students[3].Id, CourseId = courses[1].I
d, Grade = 3.9m }
        };
        context.Enrollments.AddRange(enrollments);
        context.SaveChanges();
    }
}
```

Why `Migrate()` and not `EnsureCreated()`? You already created the schema with `dotnet ef database update`. `Database.Migrate()` respects that migration history and applies anything still pending. `EnsureCreated()` is a different, incompatible path: it builds the schema with no migration record, so the next `dotnet ef database update` you run in Session 2 would fail with a "table already exists" error. Stick with migrations end to end.

Step 3: Run the Deferred Execution Experiment

Create a temporary API controller called `Controllers/TestController.cs` inside your project. This controller will host experiments to illustrate query compilation timing.

```
using Microsoft.AspNetCore.Mvc;
using System;
using System.Linq;
using TmsApi.Data;

namespace TmsApi.Controllers;

[ApiController]
[Route("api/test")]
public class TestController(TmsDbContext context) : ControllerBase
{
    [HttpGet("deferred")]
    public IActionResult TestDeferred()
    {
        Console.WriteLine("\n>>> STEP 1: Building the query object (no
database contact)...");
        var query = context.Students.Where(s => s.GPA >= 3.0m);

        Console.WriteLine(">>> STEP 2: Appending a sorting clause...");
        var orderedQuery = query.OrderBy(s => s.Name);

        Console.WriteLine(">>> STEP 3: Materializing query into a C# Li
st...");
        var results = orderedQuery.ToList(); // Execution is triggered
here

        Console.WriteLine(">>> STEP 4: Materialization finished. List p
opulated.\n");
        return Ok(results);
    }
}
```

Start your application (`dotnet run`) and invoke this endpoint using a browser or a terminal call. Use the port your app actually printed at startup (look for the `Now listening on: http://localhost:...` line it is set by `launchSettings.json` and is not always `5000`):

```
curl http://localhost:5000/api/test/deferred
```

Look closely at your application console window. Notice the output order.

Observer Challenge: Verify that the database queries and connection logs occur exactly *between* Step 3 and Step 4. This demonstrates that LINQ queries acting on `IQueryable` are merely instruction sets (recipes) and do not run until you request the final collection (materialization).

Step 4: Run the SQL Translation Failure Experiment

Add a helper C# method and a new testing endpoint to your `TestController` class to see what happens when C# functions cannot be translated to SQL:

```
    // Non-translatable helper method
    private static bool IsHonorRoll(decimal gpa)
    {
        return gpa >= 3.5m;
    }

    [HttpGet("translation-fail")]
    public IActionResult TestTranslationFail()
    {
        Console.WriteLine("\n>>> STEP 1: Running non-translatable quer
y...");
        try
        {
            var students = context.Students
                .Where(s => IsHonorRoll(s.GPA)) // EF Core does not kno
w how to map this method to SQL
                .ToList();
            return Ok(students);
        }
        catch (Exception ex)
        {
            Console.WriteLine($">>> EXCEPTION CAUGHT: {ex.Message}\n");
            return BadRequest(new { Message = ex.Message });
        }
    }
```

Invoke this endpoint in your terminal:

```
curl http://localhost:5000/api/test/translation-fail
```

The application will catch an `InvalidOperationException` stating: `The LINQ expression '...' could not be translated. Either rewrite the query in a form that can be translated, or switch to client evaluation...`

Why did this fail? EF Core parses your LINQ code into an Abstract Syntax Tree (AST) to generate SQL. Because your custom method `IsHonorRoll` is compiled C# IL, Npgsql cannot translate it into a PostgreSQL function.

How to Resolve This: You have two primary resolution patterns depending on performance requirements: 1. **Server-Side Evaluation (Preferred)**: Replace the custom helper method with inline logic that EF Core can parse directly: `csharp var students = context.Students.Where(s => s.GPA >= 3.5m).ToList();` *Verify in log*: This sends a clean, filtered query: `SELECT ... FROM "Students" WHERE "GPA" >= 3.5`. 2. **Client-Side Evaluation**: Pull the dataset into memory using `.AsEnumerable()` before applying C# logic: `csharp var students = context.Students .AsEnumerable() // Pulls all rows into application RAM .Where(s => IsHonorRoll(s.GPA)) .ToList();` *Warning*: While this works, look at the generated SQL in your console. It sends `SELECT ... FROM "Students"`, pulling the entire table across the network. If your database grows to millions of records, this will degrade application performance.

Step 5: Solve the Registrar's Business Queries

Now that you understand materialization and translation, write endpoints to solve the four registrar requests. Ensure that the processing is fully translated and executed by the database.

Add these queries to a new controller or a reporting service. Write your queries using the syntax patterns below and check the console logs to confirm that filters, sorting, and aggregations are present in the SQL output:

1. **How many active students have GPA >= 3.0?**
    - **C#**:

      ```
      var count = await context.Students
          .Where(s => s.IsActive && s.GPA >= 3.0m)
          .CountAsync();
      ```

    - **SQL Check**: Ensure you see `SELECT COUNT(*)::int4 FROM "Students" AS s WHERE s."IsActive" AND s."GPA" >= 3.0`.
2. **Which courses have the most enrollments, sorted descending?**
    - **C#**:

      ```
      var list = await context.Courses
          .Select(c => new
          {
              c.Title,
              EnrollmentCount = c.Enrollments.Count
          })
          .OrderByDescending(x => x.EnrollmentCount)
          .ToListAsync();
      ```

- **SQL Check**: Look for `ORDER BY` and count calculations happening in SQL, not C#.

3. **What is the average GPA per course?**
   - **C#**:

```
var list = await context.Enrollments
    .GroupBy(e => e.Course.Title)
    .Select(g => new
    {
        Course = g.Key,
        AverageGPA = g.Average(e => e.Student.GPA)
    })
    .ToListAsync();
```

   - **SQL Check**: Look for `GROUP BY` and the `AVG` aggregation function in the log.

4. **Which students have zero enrollments?** (Show both patterns)
   - **Approach A (Using Subquery)**:

```
var list = await context.Students
    .Where(s => !s.Enrollments.Any())
    .Select(s => s.Name)
    .ToListAsync();
```

   - **Approach B (Using EF Core 10 LeftJoin)**:

```
var list = await context.Students
    .LeftJoin(context.Enrollments,
        s => s.Id,
        e => e.StudentId,
        (s, e) => new { s, e })
    .Where(x => x.e == null)
    .Select(x => x.s.Name)
    .ToListAsync();
```

   - **SQL Check**: Observe the output queries. Approach A uses `NOT EXISTS (SELECT 1 FROM "Enrollments" ...)`, while Approach B outputs a `LEFT JOIN ... WHERE ... IS NULL`.

Extended Exercise (Stretch): Wire `Assessment` and `Certificate` into the Database

You already wrote `Assessment` and `Certificate` in Step 1, but they are not tables yet. Nothing in `TmsDbContext` references them, so EF Core never put them in the model. The registrar still cannot record a mark or issue a certificate. Your job is to wire them in and migrate them, applying the exact loop from Exercise 1 with **no commands handed to you beyond the names**.

This proves a point worth internalising: *defining a class is not the same as creating a table.* Membership in the EF Core model is what counts.

Your task

1. **Register both entities** as `DbSet` properties on `TmsDbContext`, following the existing `Set<Student>()` / `Set<Course>()` / `Set<Enrollment>()` pattern. (Optional, and a good check of your understanding: add `ICollection<Assessment>` to `Course`, and `ICollection<Certificate>` to `Student` and `Course`, so the relationships read from both sides. Notice this does **not** change the schema. The foreign keys already live on `Assessment`/`Certificate`.)

2. **Generate a new migration** *not* a second `InitialCreate`. Give it a descriptive name:

   ```
   dotnet ef migrations add AddAssessmentsAndCertificates
   ```

3. **Inspect it before applying.** Open the new migration and confirm:

   - Only the two new tables appear in `Up()`; your existing three tables are untouched.
   - The foreign keys point where you expect (`Assessments.CourseId` → `Courses`; `Certificates.StudentId` → `Students` and `Certificates.CourseId` → `Courses`), and `Down()` drops the new tables before anything they depend on.

4. **Apply and verify:**

   ```
   dotnet ef database update
   psql -U postgres -d TmsDb -c "\dt"
   ```

   You should now see five tables instead of three.

Hints, not solutions

- A `DbSet<T>` is the single line that pulls a class into the model. If your migration comes back **empty**, you forgot to add the `DbSet` (or saved nothing). If it tries to recreate existing tables, you accidentally ran `InitialCreate` again instead of a new migration name.
- The foreign keys are already on the entities from Step 1 (`CourseId`, `StudentId`), so EF will infer the relationships by convention. you should not need any extra configuration here.
- If the migration shows changes you did not intend, run `dotnet ef migrations remove`, fix the cause, and regenerate before touching the database.

Done when

- `psql ... "\dt"` lists `Assessments` and `Certificates` alongside the original three tables.
- You can point to each foreign key in your migration's `Up()` method and say which table it links to.

Why This Session Ends Here

In this session, you established the persistence layer foundation: 1. **A working database context mapping real Postgres tables**. Your TMS database now persists records across API restarts. 2. **Comprehensive SQL Logging**. You can inspect the queries generated by your C# code and identify sub-optimal execution paths before shipping. 3. **LINQ Translation discipline**. You understand that `IQueryable` expressions execute lazily at the database level, and how to avoid client-side evaluation performance traps.

In the next session, you will configure schema specifics, set up paging for large record lists, and use configurations to define relationship constraints.

Session 1 Checkpoint

Before proceeding to Session 2, confirm that: - [ ] `dotnet ef database update` executes successfully and you can view the tables in PostgreSQL. - [ ] You can explain what both `Up()` and `Down()` methods do inside your initial migration file. - [ ] The SQL log verifies that filters, aggregates, sorting, and joins run on the database server. - [ ] You can explain why calling `.ToList()` before `.Where()` poses a performance concern in database-backed systems. - [ ] *(Stretch)* If you took the extended exercise, `\dt` shows `Assessments` and `Certificates`, and you applied them through a named migration. not `EnsureCreated()`.